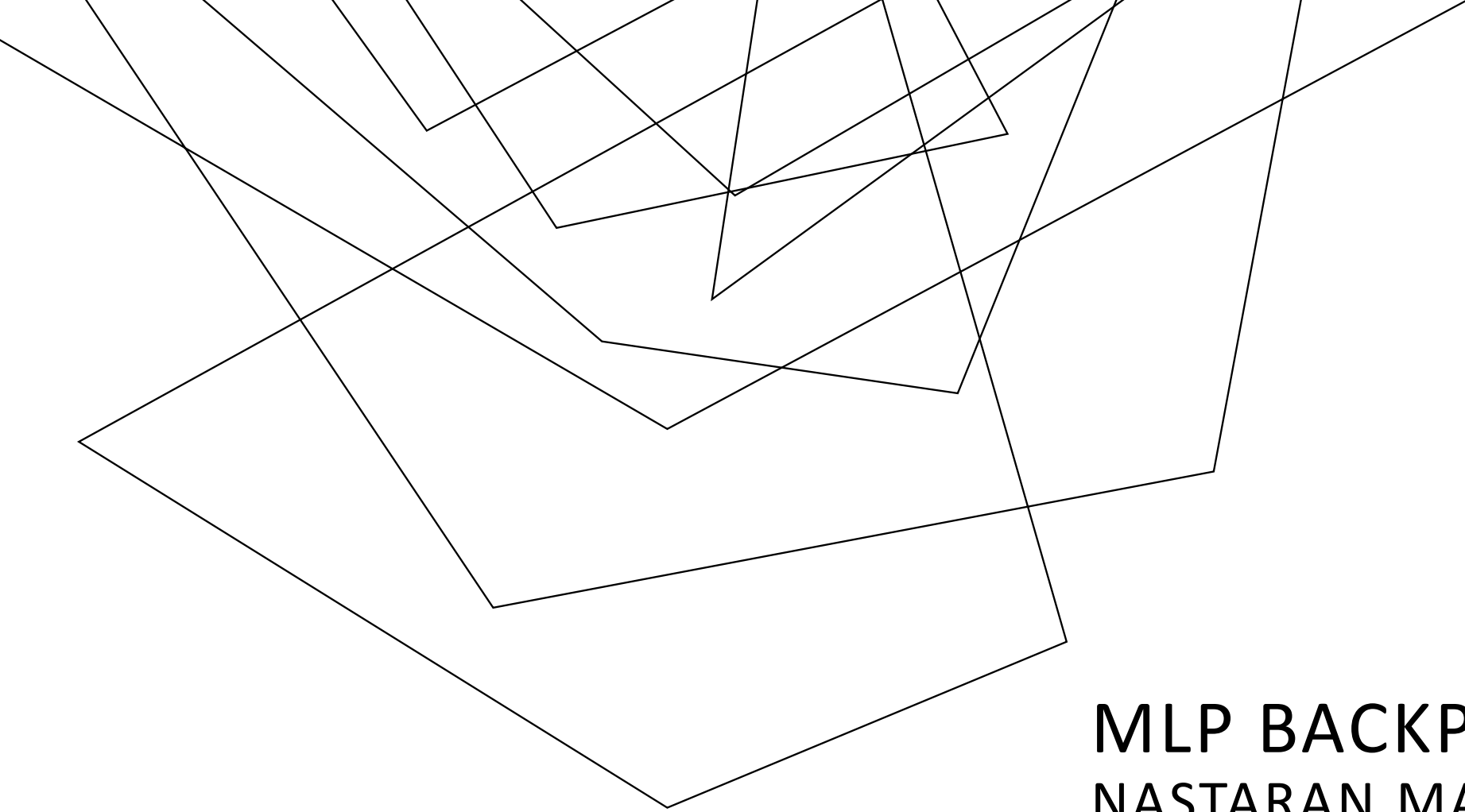


Abstract geometric lines in the top-left corner of the slide, consisting of several thin black lines forming overlapping, irregular shapes.

MLP BACKPROPAGATION

NASTARAN MAKOUI
2501393

DATA: HOW IT WAS COLLECTED, PARTITIONED AND PROCESSES

- As expected, the data was collected by playing the game and was saved in "ce889_dataCollection", the raw dataset file had:
 - 1) x_target , y_target representing positions
 - 2) Velocities for x and y , which were gathered by playing the game and later ignored due to noise
- But what does the data actually represent:
 - 1) It does represents x_target and y_target , which are essentially the rocket's distance from the landing point
 - 2) v_x , v_y , which are desired horizontal and vertical from my mathematical controller and are not the noisy ones collected by playing the game.
- The data was split in 3 section of training (70%), validation (15%) and test (15%). Training was used to train the weights, validation for checking overfitting and test for the final evaluation which needed to be unbiased
- For the inputs, game resolution was used in this game as a way to measure distances and control the rocket, also the diagonal distance of the game screen was used as a normalization scale to make x and y the same scale.
- For the output(targets), I used a mathematical controller:
 - 1) Horizontal, choose vx_max based on $|x_target|$ (speed 6.0 if far, 5.0 medium, 3.5 close), then proportional control + clamping
 - 2) Vertical enforce minimum safe altitude ($BASE_ALT = 120$), and altitude gain ($ALT_GAIN = 0.35$) so the rocket flies higher when far from the pad.

ARCHITECTURE AND TOPOLOGY

The architecture used was : 2 -> 6 -> 2

Input layer: 2 neurons

Hidden layer: 6 neurons, used sigmoid activation function

Output layer: 2 neurons, used linear activation function

The reason sigmoid has been used in the hidden layer is because it squashes everything between 0 and 1, causing a smooth nonlinearity, it's derivative is also easy to compute and is used in backpropagation.

But why did we use linear in the output layer then? Because in the code, outputs are linear neurons so they can represent any real value also the *normalization and de-normalization* of velocities is done outside the neuron, which keeps the math clearer.

One of the main sources and reason to use sigmoid for the hidden layer and linear for output layer was Example 2, uploaded on moodle:

In which it was shown that for output layer the linear function was used while very specifically it was mentioned that hidden layer uses sigmoid.

(a) The forward pass equations could be written as follows:

$$\begin{aligned} y_k &= \varphi \left[\sum_{i=0}^n w_{ki} h_i \right] \\ &= \varphi \left[\sum_{i=0}^n w_{ki} \varphi^h \left(\sum_{j=0}^m w_{ij}^h x_j \right) \right] \end{aligned}$$

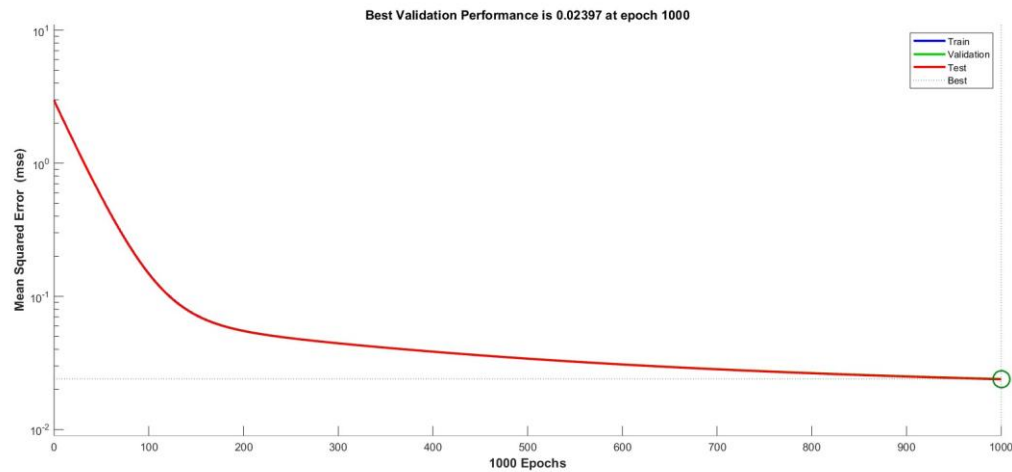
For the hidden the Neuron, the activation function is sigmoid hence,

$$\varphi(v) = \frac{1}{1 + \exp(-\lambda v)} \quad \varphi'(v) = \lambda \varphi(v)(1 - \varphi(v))$$

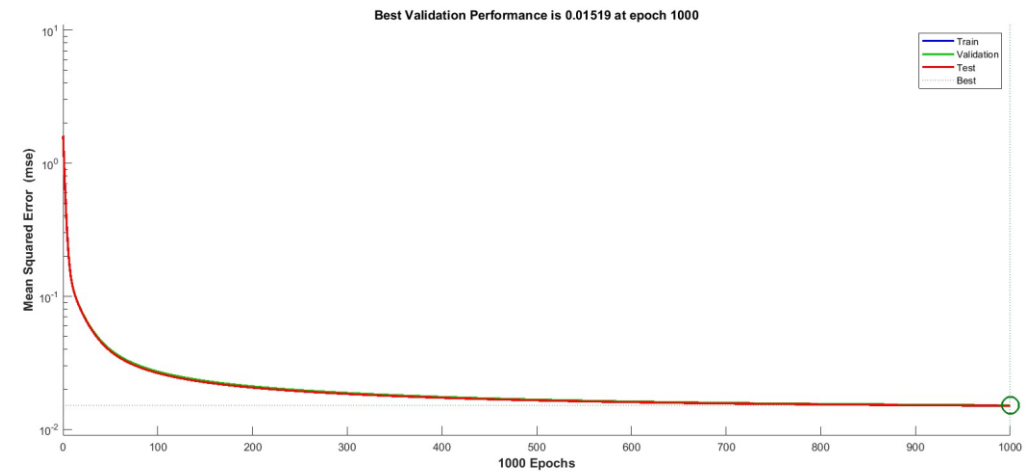
SIZE COMPARISON

In order to choose a number of hidden neurons that would best train my neural network I researched to see what where the experts' opinion on the matter was and I ended up using 3 sources for this project:

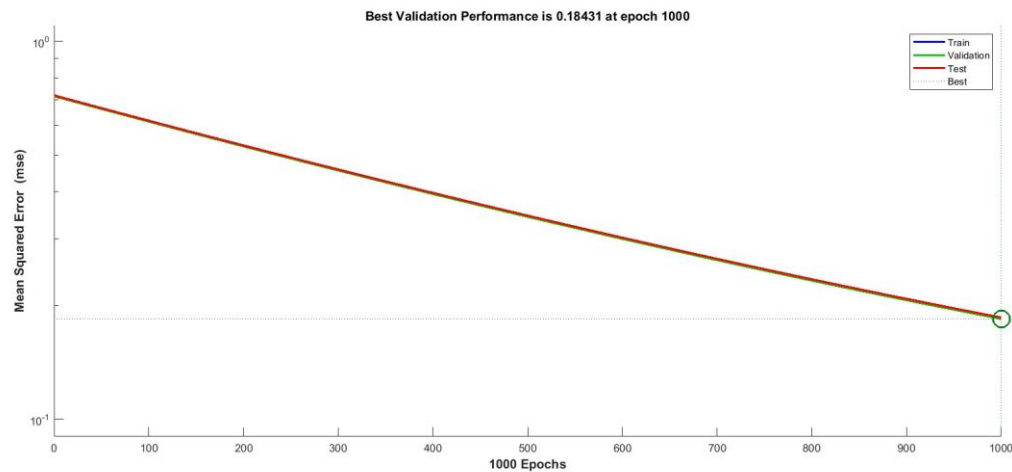
- The Number of Hidden Layers by Heaton Research (2017): Jeff Heaton expertly explains that choosing the right number of neurons is a balance between underfitting and overfitting, as too few neurons will lead to network not being able to detect the underlying patterns and too many neurons will cause the network to memorize and learn from noises. Heaton also suggest a series of rules of thumbs which all supported my decision of 6 neurons, first one being a network of 2 inputs and 2 outputs needs a small hidden layer and as you will see in the next slides, my experiments showed 4 neurons to be underfitting while 10 was slightly overfitting. This matches Heaton's guidance that the *final choice should come from empirical testing after starting with a reasonable range*.
- Second paper used in this project is an interesting thesis I came across during my research called "Using upper layer weights to efficiently construct and train feedforward neural networks executing backpropagation" (Gage, 2011), this thesis also agrees with the points made in Heaton's research and believes hidden layers with 4 neurons will often result in underfitting while layers with 12 and more neurons will usually result in overfitting, so it is important to find that sweet spot in-between. The thesis further validates my method because it explicitly states that trial-and-error comparison is a legitimate and academically recognized architecture-selection method when training small MLPs.
- The last source that I used for this part was the book "Deep Learning" by Bengio, Goodfellow & Courville which was in our reading list for this module, in chapter 6: Deep Feedforward Networks, it is stated simple problems usually require narrow layers and wider layers increase overfitting risks and they also mention that hidden layer size is a hyperparameter determined by experimentation



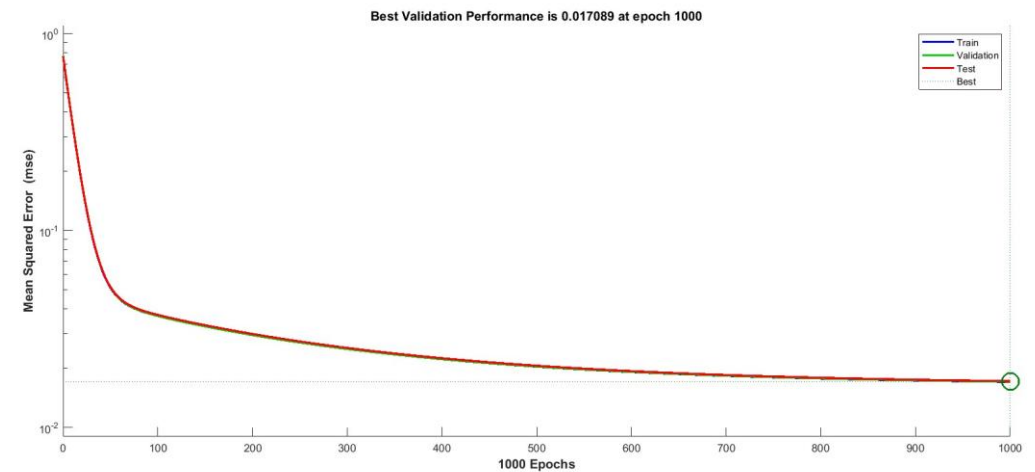
Validation Performance: 0.023, hidden neurons = 8
learning rate = 0.01



Validation Performance: 0.015, hidden neurons = 10
learning rate = 0.12



Validation Performance: 0.18, hidden neurons = 4
learning rate = 0.001



Validation Performance: 0.017, hidden neurons = 6
learning rate = 0.03

PARAMETERS

As seen in the slide 5, I experimented with different hyperparameters and the best combination was 6 hidden neurons and a learning rate of 0.03. similarly to hidden neurons, I reached the number for learning rate through research as well.

- The source that helped me the most was the paper “Efficient Backprop” by LeCun et al. LeCun emphasizes that step sizes matter a lot and taking too large of a step causes instability and too small of a step slows the learning process. It can be seen in the previous slide’s graphs as well, in the case with the highest learning rate, we see higher instability early in the training and a high learning rate like 0.12 is borderline unusable for sigmoid. And the one with the smallest learning rate had produced the highest error and is extremely slow
- LeCun also justifies the use of sigmoid function in the hidden layer by stating that the logistic function, also known as sigmoid function, is commonly used and inputs should be normalized.
- Another source that helped me was Geoffrey Hinton’s lectures available online, specially his lecture on “Overview of mini batch gradient descent”. My training method uses online gradient descent, because I update the weights after every individual training example, not after batches or full epochs. Which was something explained in Prof.Hinton’s lecture.

- The combination of learning rate = 0.03 and hidden neurons = 6 gave me a mse of 0.017, which was the best I had achieved, I also managed to get my rocket to land with learning rate = 0.01 and hidden neurons = 8 which produced mse of 0.023, and arguably the landing with the second combination is smoother but the scores from the game is much lower and the mse is also arguably much higher.
With the combination of 6 and 0.03, the rocket hovers over the landing spot and takes it's time to land but get's a much better score and not mention a much better mse!
- My MLP uses online gradient descent without momentum, which is equivalent to having a momentum value of zero. Since the network is small and the gradients are stable, momentum was unnecessary—training converged smoothly without it, and the additional term would not have provided meaningful improvement.
- All neuron weights and biases are initialized with `random.uniform(-1, 1)`. The reason to do this was supported in the lecture by Hinton, as he mentioned that If two hidden units have exactly the same incoming weights, they will always get the same gradient and they can never learn different features, so I used the code `random.uniform` to break symmetry.

CONTROLLER & NORMALISATION CONSTANTS

As mentioned before, this code heavily relied on the screen width and height. One of the most important things was the consistency between training and prediction, which is *crucial*; it is stressed in both my comments and the literature.

For Horizontal:

if it is too far , meaning $|x_target| > 300 \rightarrow vx_max = 6$, which means we boost the speed

if $|x_target| > 200 \rightarrow vx_max = 5$, which is medium speed

else: $vx_max = 3.5$ meaning we go gentle to avoid the rocks

The horizontal values are also clamped so the network does not see impossible or unstable values.

Max speed is also 8.

For Vertical:

BASE_ALT = 120 pixels:

About 20% of screen height $\rightarrow$ high enough to avoid the tall rocks but not too high to make landing difficult.

ALT_GAIN = 0.35:

Controls how much extra altitude we want when the lander is horizontally far from the pad.

Larger value $\rightarrow$ more aggressive climb, smaller value $\rightarrow$ risk of colliding with obstacles.

Vertical speed is also clamped to a safe range so the descent stays smooth and controllable.

RMSE & EARLY STOPPING BEHAVIOR

The code prints MSE every 10 epochs:

Well RMSE is simply $\sqrt{MSE}$ and you can show a line graph: x-axis = epoch, y-axis = RMSE.

Validation and test MSE/RMSE is each 15%

Stopping criterion (what I actually used)

I used a fixed maximum of 100 epochs as a simple early-stopping rule.

Justification:

From the printed MSE values, the improvement per 10 epochs gets smaller over time → diminishing returns.

100 epochs is enough to reach a low error without over-training forever.

```
ment Code/Assignment Code/lander_mlp_neuron_based.py"
Loading dataset...
Training samples: 60337
Validation samples: 12929
Test samples: 12930
Epoch 0 | MSE = 0.005480
Epoch 10 | MSE = 0.002544
Epoch 20 | MSE = 0.002008
Epoch 30 | MSE = 0.001588
Epoch 40 | MSE = 0.001538
Epoch 50 | MSE = 0.001523
Epoch 60 | MSE = 0.001510
Epoch 70 | MSE = 0.001498
Epoch 80 | MSE = 0.001487
Epoch 90 | MSE = 0.001476
Epoch 99 | MSE = 0.001466
Weights saved to lander_neuron_weights.txt
Validation MSE = 0.001498
Test MSE = 0.001493
```

A series of white, overlapping geometric lines and polygons on a black background, located on the left side of the slide.

GOING BEYOND

I don't think I have done anything beyond the scope of what the project asked for, especially since I am not coming from a computer science background and this module has been extremely challenging for me. But I am genuinely pleased with the mathematical controller I was able to create, even though I did have sources of inspiration. All of these sources have been sent to the lab assistants. For the mathematical controls, understanding code from different open-source materials helped me massively, and the research papers I found — all of which I have named and submitted — were a great help. I am extremely happy that I was able to find them and learn from them.